

Trường Đại học Giao Thông Vận Tải thành phố Hồ Chí Minh
Viện Công nghệ thông tin và Điện, Điện tử

Chương 1.

Tổng quan về Học tăng cường (Reinforcement Learning – RL)

Tiến sĩ Nguyễn Thị Khánh Tiên
tiennstk@ut.edu.vn



Giới thiệu về Học tăng cường

Học tăng cường (Reinforcement Learning – RL) là một nhánh của Machine Learning, trong đó **tác nhân (agent)** học cách đưa ra quyết định bằng cách tương tác với **môi trường (environment)** để **tối đa hóa tổng phần thưởng (cumulative reward)**.

RL thường được mô hình hóa bằng **Markov Decision Process (MDP)**.

Điểm khác biệt giữa RL với SL và UL

Loại học máy	Cách thức hoạt động	Ví dụ
Có giám sát (Supervised)	Học từ dữ liệu đã được gán nhãn (có đáp án sẵn).	Nhận diện khuôn mặt.
Không giám sát (Unsupervised)	Tìm cấu trúc ẩn trong dữ liệu không nhãn.	Phân nhóm khách hàng.
Học tăng cường (RL)	Học từ trải nghiệm và phản hồi.	Robot tìm đường trong mê cung.

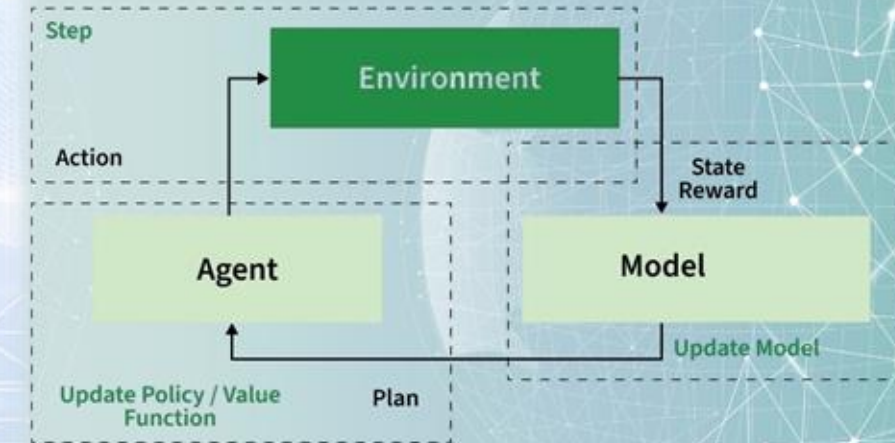
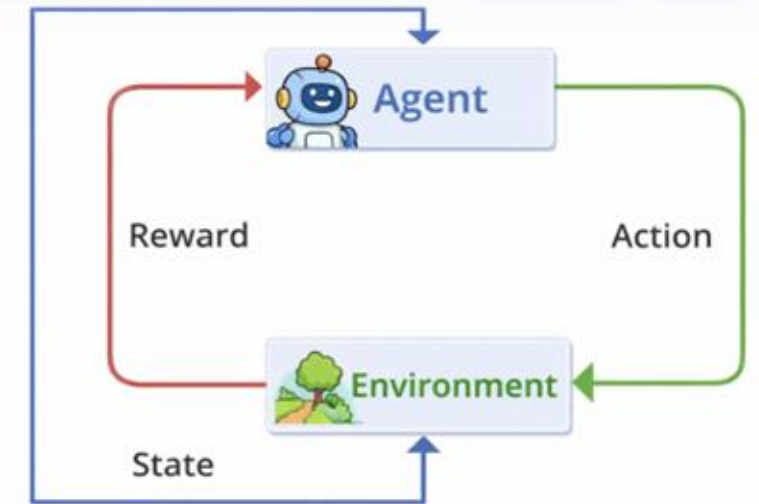
Các thành phần cốt lõi và khái niệm quan trọng của RL

1. **Tác nhân (Agent):** thực thể học tập và ra quyết định (ví dụ: chú chó, phần mềm chơi game).
2. **Môi trường (Environment):** Thế giới nơi Agent hoạt động (ví dụ: ngôi nhà, bàn cờ).
3. **Trạng thái (State – s):** Tình huống hiện tại của Agent trong môi trường.
4. **Hành động (Action – a):** Hành động agent có thể thực hiện.
5. **Phần thưởng (Reward – r):** Tín hiệu phản hồi từ môi trường để đánh giá hành động đó tốt hay xấu:
 - Dương $\rightarrow$ tốt
 - Âm $\rightarrow$ xấu
6. **Chiến lược (Policy – $\pi(a | s)$)** Chiến lược chọn hành động dựa trên trạng thái, giúp Agent quyết định nên làm gì dựa trên trạng thái hiện tại.
7. **Giá trị (Value Function)**
 - $V(s)$: giá trị kỳ vọng khi bắt đầu từ state s
 - $Q(s, a)$: giá trị khi chọn action a tại state s
8. **Discount factor – γ**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

$\gamma \in [0, 1]$

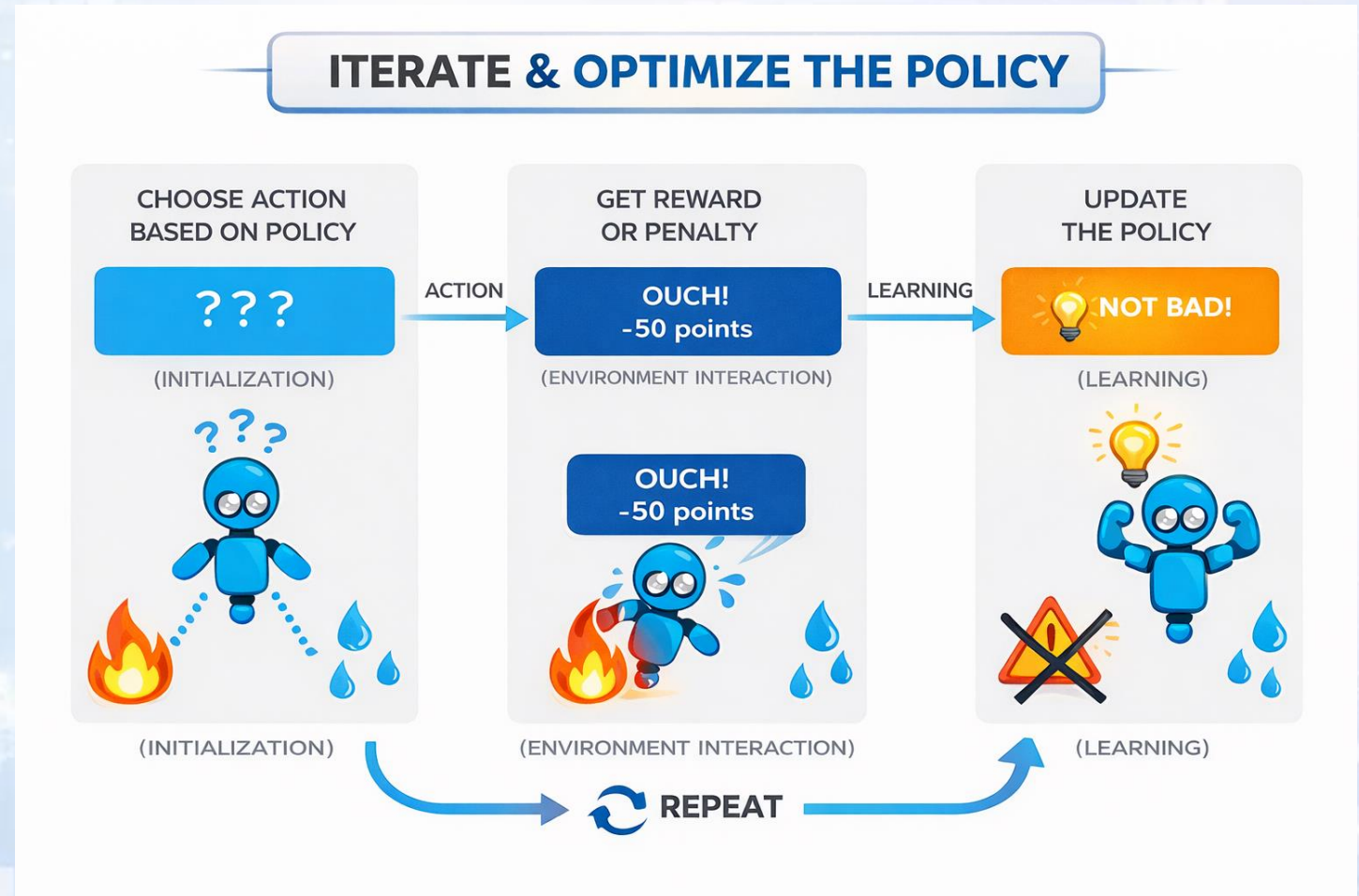
Điều chỉnh mức độ quan tâm đến phần thưởng tương lai.



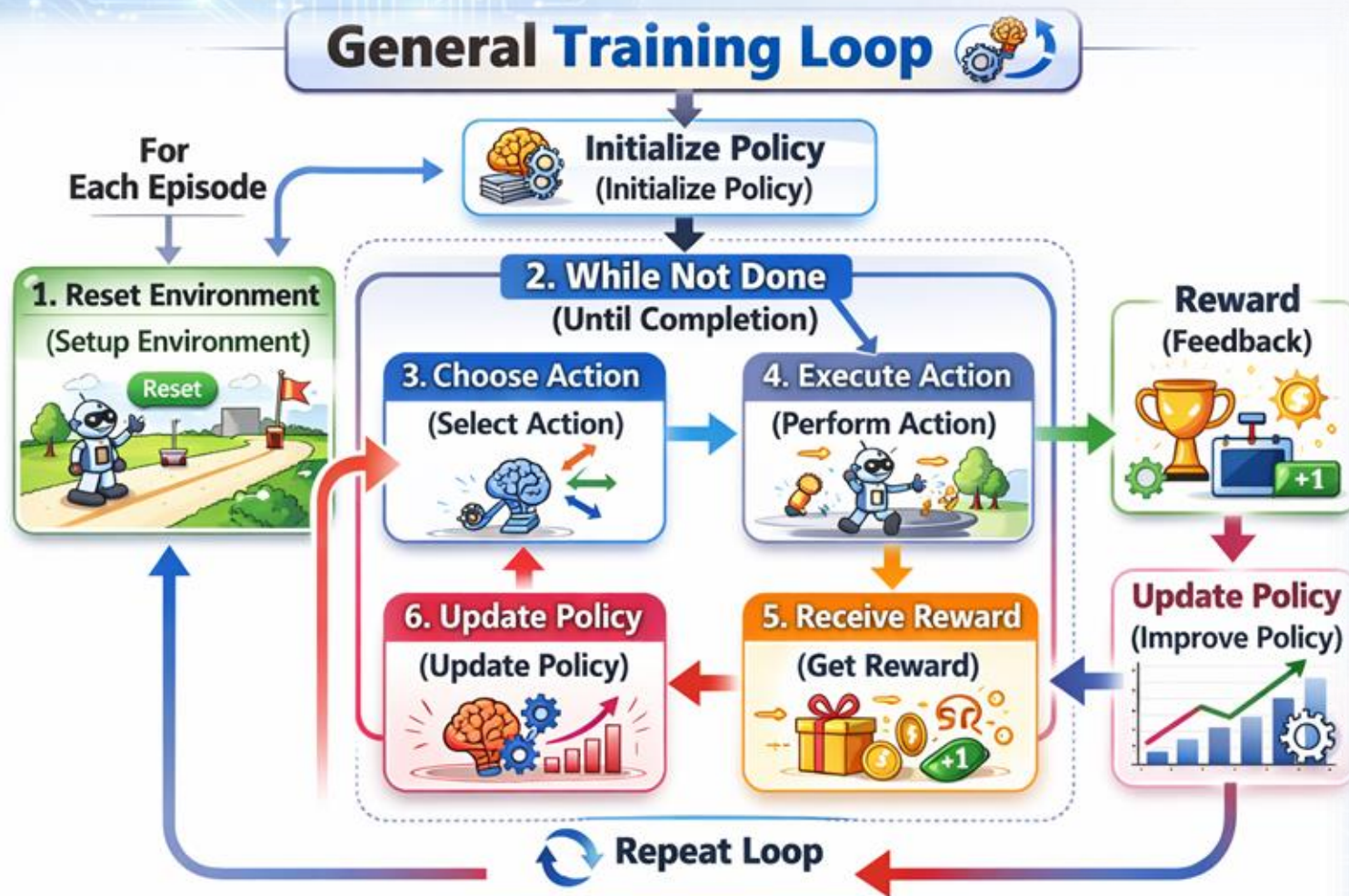
PhD Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Quy trình làm việc của RL

- **Khởi tạo:** Tác nhân bắt đầu ở trạng thái ban đầu của môi trường.
- **Lựa chọn hành động:** Tác nhân chọn một hành động dựa trên chính sách hiện tại của mình.
- **Tương tác với môi trường:** Tác nhân thực hiện hành động đã chọn, môi trường chuyển sang trạng thái mới.
- **Phần thưởng:** Tác nhân nhận được phần thưởng tương ứng với sự chuyển trạng thái đó.
- **Học tập:** Tác nhân cập nhật chính sách dựa trên phần thưởng nhận được nhằm cải thiện hiệu suất.
- **Lặp lại:** Quá trình này được thực hiện lặp đi lặp lại, giúp tác nhân dần nâng cao khả năng ra quyết định.



Quy trình làm việc của RL



Vòng lặp huấn luyện tổng quát:

I> Initialize policy (khởi tạo chính sách)

II>for episode (cho mỗi episode):

1. reset environment (thiết lập môi trường)
2. while not done (trong khi chưa hoàn thành):
 3. choose action (chọn hành động)
 4. execute action (thực hiện hành động)
 5. receive reward (nhận phần thưởng)
 6. update policy (cập nhật chính sách)

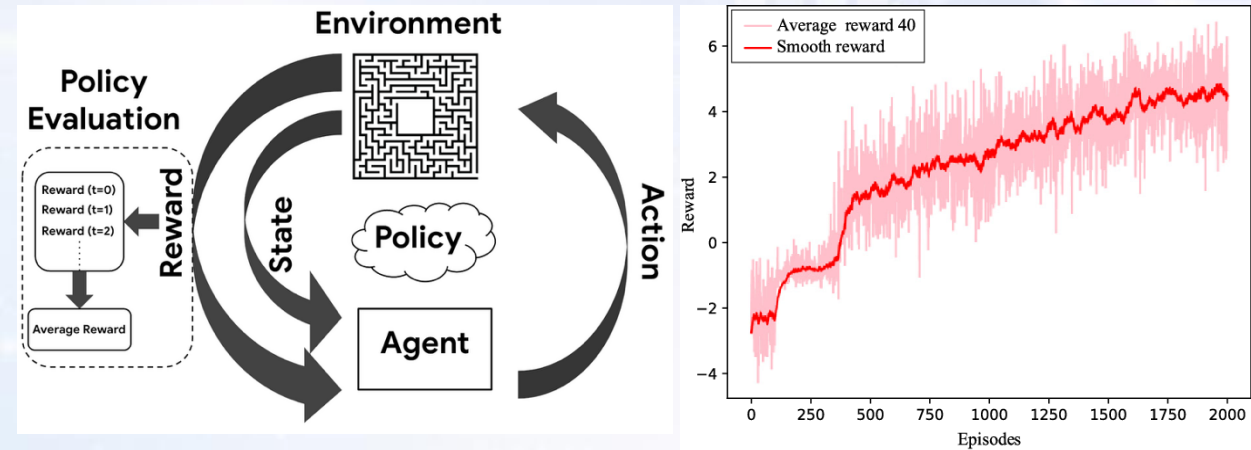
PhD Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Đánh giá mô hình RL

Đánh giá một mô hình RL phức tạp hơn nhiều so với học có giám sát (Supervised Learning). Thay vì chỉ nhìn vào độ chính xác (accuracy), chúng ta phải đánh giá khả năng tối ưu hóa phần thưởng và sự ổn định của tác nhân (agent) trong môi trường động.

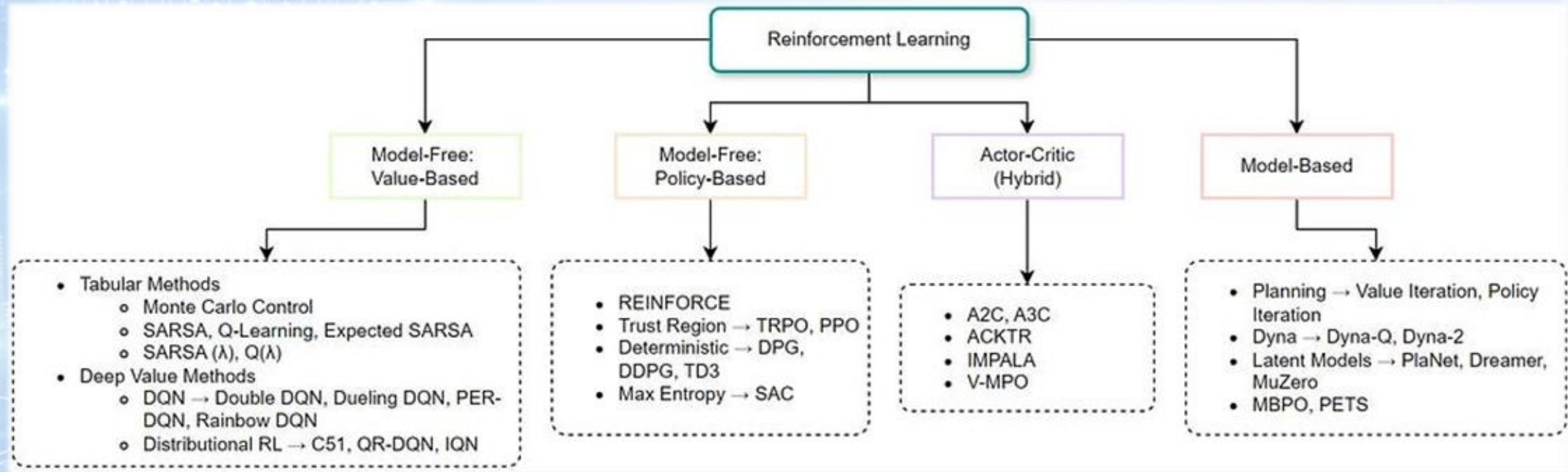
Các tiêu chí và phương pháp cốt lõi để đánh giá một mô hình RL:

- Các chỉ số hiệu suất chính (Key Metrics): tổng phần thưởng tích lũy (Cumulative Rewards/Return), độ dài Episode (Episode Length, giá trị hội tụ (Convergence))
- Khả năng hội tụ và Độ ổn định (Stability & Convergence): độ lệch chuẩn của phần thưởng (nếu cùng một policy nhưng cho kết quả quá khác biệt giữa các lần chạy, mô hình đó chưa đáng tin cậy); Policy Loss & Value Loss
- Khả năng tổng quát hoá (Generalization): kiểm tra trên môi trường lạ, Zero-shot/Few-shot Transfer (đưa agent vào một bản đồ hoàn toàn mới mà nó chưa từng thấy trong lúc huấn luyện)
- Hiệu quả sử dụng mẫu (Sample Efficiency): số lượng bước tương tác (Steps- để đạt được một mức phần thưởng X, mô hình cần bao nhiêu triệu bước tương tác với môi trường), các thuật toán Off-policy (như SAC, DQN) thường có hiệu quả sử dụng mẫu tốt hơn các thuật toán On-policy (như PPO).



Phương pháp	Mục tiêu	Khi nào sử dụng
Exploitation vs Exploration	Kiểm tra xem agent có bị kẹt ở tối ưu địa phương không.	Giai đoạn đầu huấn luyện.
Robustness Testing	Thêm nhiễu vào quan sát (observation) để thử độ bền.	Trước khi triển khai thực tế.
Mean Cumulative Reward	Đánh giá hiệu suất trung bình.	Xuyên suốt quá trình học.

Phân loại trong RL



1. Phương pháp dựa trên giá trị (Value-Based Methods)

Các phương pháp này ước lượng **giá trị** của **mỗi trạng thái** hoặc **cặp trạng thái – hành động** trong môi trường. Tác nhân học cách lựa chọn hành động tốt hơn bằng cách so sánh các giá trị này.

2. Phương pháp dựa trên chính sách (Policy-Based Methods)

Trong các phương pháp này, tác nhân **học trực tiếp chính sách**, tức là học một hàm ánh xạ từ trạng thái sang xác suất chọn mỗi hành động. Cách tiếp cận này đặc biệt hiệu quả trong: Không gian hành động có số chiều cao, không gian hành động liên tục

3. Phương pháp dựa trên mô hình (Model-Based Methods)

Các phương pháp này xây dựng **mô hình của môi trường**. Tác nhân học cách môi trường vận hành (chuyển trạng thái và phần thưởng), sau đó sử dụng mô hình này để ra quyết định.

Ưu điểm: Có thể tiết kiệm dữ liệu hơn (sample efficient)

Hạn chế: Yêu cầu mô hình môi trường phải đủ chính xác

4. Mô hình Actor-Critic (Hybrid).

Kết hợp ước lượng giá trị và tối ưu hóa chính sách trực tiếp để đạt được sự ổn định và hiệu quả.

Ứng dụng của RL:

1. Robotics (Người máy)

Điều khiển robot thực hiện các nhiệm vụ như gấp vật thể, di chuyển trong môi trường phức tạp, điều hướng tự động hoặc thao tác chính xác trong công nghiệp.

2. Chơi game (Game Playing)

Phát triển các tác nhân (agent) có khả năng chơi và vượt qua con người trong các trò chơi như cờ vua, cờ vây (Go) và trò chơi điện tử.

3. Tài chính (Finance)

Ứng dụng trong giao dịch thuật toán (algorithmic trading), quản lý danh mục đầu tư và tối ưu hóa chiến lược đầu tư.

4. Y tế (Healthcare)

Xây dựng phác đồ điều trị cá nhân hóa cho bệnh nhân và hỗ trợ quá trình khám phá, phát triển thuốc mới.

5. Hệ thống gợi ý (Recommendation Systems)

Đề xuất sản phẩm, nội dung hoặc dịch vụ phù hợp với sở thích và hành vi của người dùng.

Các thách thức của RL:

1. Hiệu quả mẫu (Sample Efficiency)

Các thuật toán RL thường cần một lượng lớn dữ liệu tương tác để học hiệu quả, điều này gây tốn kém thời gian và chi phí.

2. Bài toán Cân bằng Khám phá – Khai thác (Exploration–Exploitation Dilemma)

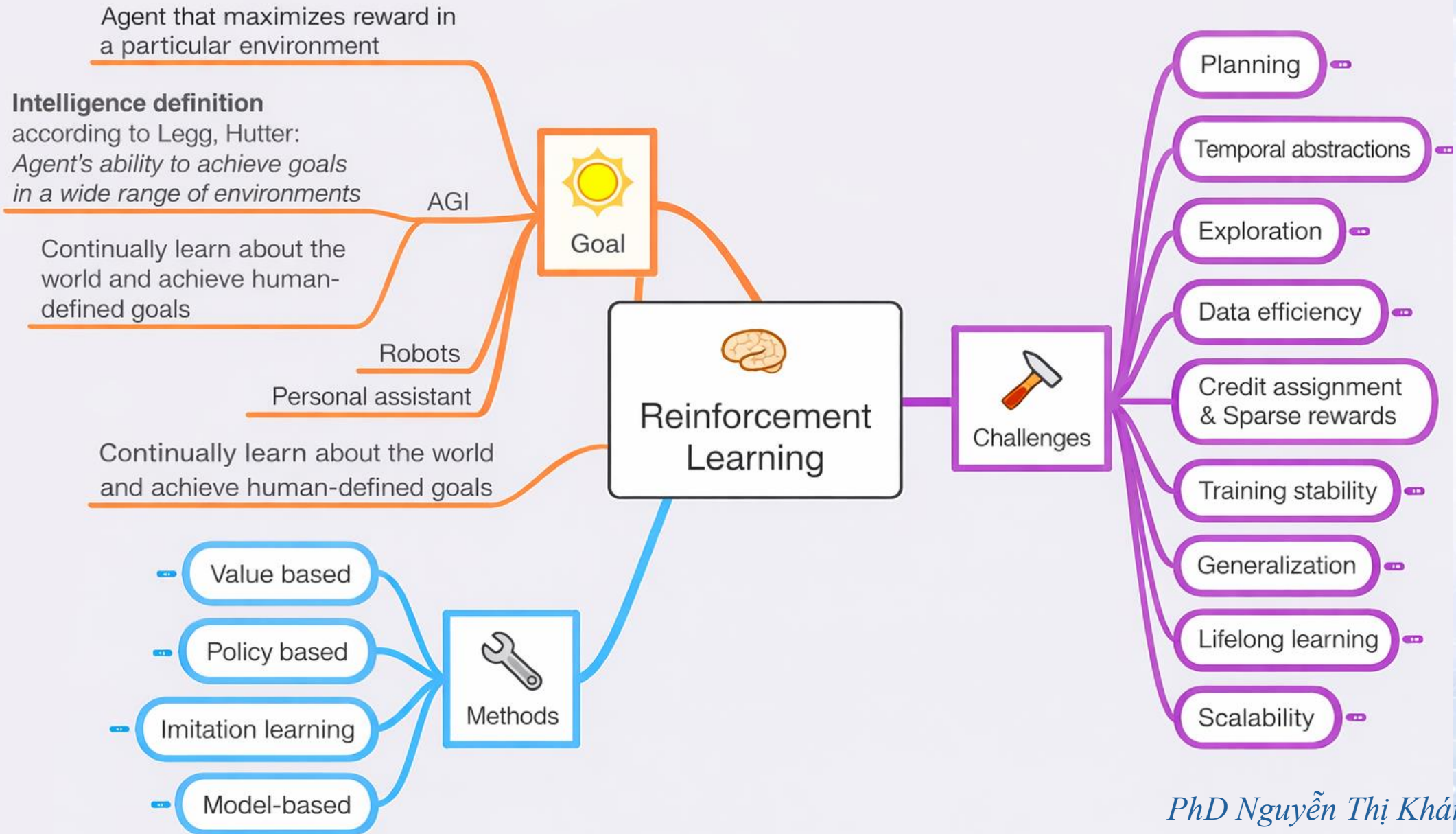
Việc lựa chọn giữa khám phá các hành động mới và khai thác những hành động đã biết là tối ưu vẫn là một vấn đề khó.

3. Gán công trạng (Credit Assignment Problem)

Khó xác định chính xác hành động nào trong chuỗi hành động đã đóng góp vào phần thưởng nhận được.

Hướng nghiên cứu tương lai của RL:

1. Nghiên cứu trong tương lai tập trung vào:
2. Phát triển các thuật toán hiệu quả hơn về dữ liệu.
3. Cải thiện khả năng học với ít mẫu hơn.
4. Mở rộng ứng dụng RL vào các bài toán thực tế có độ phức tạp cao và môi trường không chắc chắn.



Các thư viện Python sử dụng cho RL

1. Môi trường mô phỏng (Environments)

- Gymnasium (trước đây là OpenAI Gym)
- PettingZoo: Tương tự như Gymnasium nhưng dành cho Multi-Agent RL (nhiều AI tương tác hoặc đối đầu với nhau).
- PyBullet / MuJoCo: Các bộ máy vật lý chuyên dụng để mô phỏng robot và các chuyển động cơ học phức tạp.

2. Thư viện thuật toán (RL Frameworks)

- Stable Baselines3 (SB3): Dựa trên PyTorch, cực kỳ dễ dùng, tin cậy và có tài liệu hướng dẫn rất tốt.
- Ray Rllib: Có khả năng mở rộng cực cao, hỗ trợ huấn luyện song song trên nhiều CPU/GPU hoặc cụm máy tính (cluster).

3. Những công cụ chuyên biệt:

- Tianshou (PyTorch): Một thư viện RL cực nhanh và mô-đun hóa cao từ các nhà nghiên cứu Trung Quốc.
- ACME (DeepMind): Tập trung vào tính nghiên cứu và khả năng tái lập các thí nghiệm RL phức tạp.
- TF-Agents (Google): Thư viện chuẩn nếu bạn đang làm việc trong hệ sinh thái TensorFlow.
- Jumanji / Flashlight (JAX): Dành cho những ai muốn tốc độ xử lý "bá đạo" bằng cách tận dụng JAX để chạy RL hoàn toàn trên GPU.

Mục đích	Thư viện
Environment	Gymnasium
Deep Learning	PyTorch
Prebuilt RL	Stable-Baselines3
Visualization	Matplotlib
Logging	TensorBoard

PhD Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

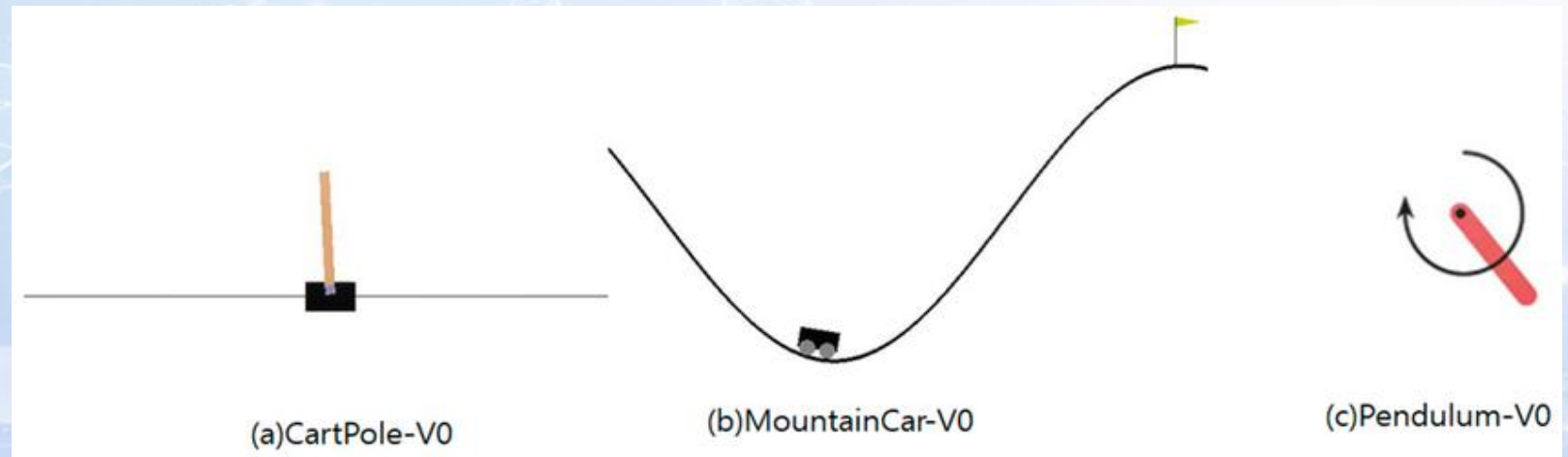
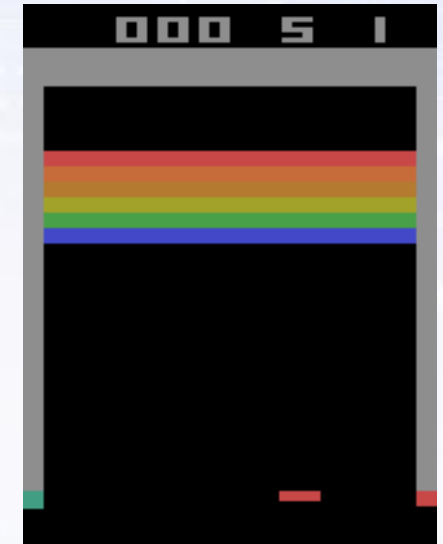
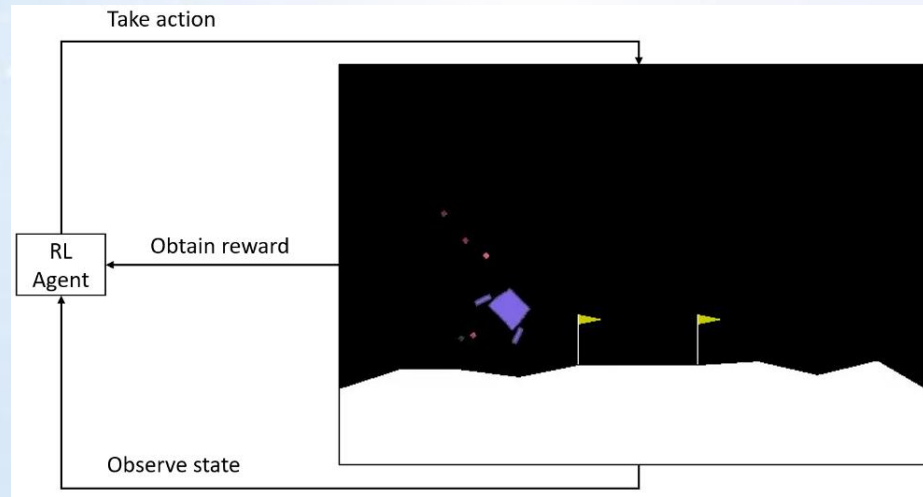
Làm quen với thư viện Gymnasium

Gymnasium là thư viện Python cung cấp các môi trường mô phỏng chuẩn cho RL. Gymnasium cung cấp:

- Các môi trường cổ điển (CartPole, MountainCar...)
- Môi trường điều khiển robot
- Game Atari
- Chuẩn API thống nhất cho RL

Các nhóm môi trường phổ biến:

- Classic Control
 - CartPole-v1
 - MountainCar-v0
 - Acrobot-v1
- Box2D
 - LunarLander-v2
 - BipedalWalker-v3
- Atari
 - ALE/Breakout-v5
 - ALE/Pong-v5



Làm quen với thư viện Gymnasium

1. Cài đặt Gymnasium

pip install gymnasium

Nếu muốn render đồ họa:

pip install gymnasium[classic-control]

2. Các thành phần cơ bản

- *Env (Environment)*: Môi trường mà agent tương tác.
- *Observation Space*: Không gian quan sát (dữ liệu đầu vào từ môi trường).
- *Action Space*: Không gian hành động (các hành động agent có thể thực hiện).
- *reset()*: Khởi tạo lại môi trường, trả về quan sát ban đầu.
- *step(action)*: Thực hiện hành động, trả về:
 - *observation*: Quan sát mới.
 - *reward*: Phần thưởng nhận được.
 - *terminated*: Tác nhân đạt trạng thái kết thúc (ví dụ: ngã).
 - *truncated*: Tác nhân bị dừng trước khi kết thúc (ví dụ: quá giới hạn thời gian).
 - *info*: Thông tin bổ sung (*debug*).

3. Quy trình chuẩn:

- `gym.make()`
- `env.reset()`
- `env.step(action)`
- `env.close()`

```
import gymnasium as gym

# 1. Khởi tạo môi trường (render_mode='human' để xem trực quan)
env = gym.make("CartPole-v1", render_mode="human")

# 2. Reset môi trường
observation, info = env.reset()

for _ in range(1000):
    # 3. Chọn một hành động ngẫu nhiên (0: Đẩy trái, 1: Đẩy phải)
    action = env.action_space.sample()

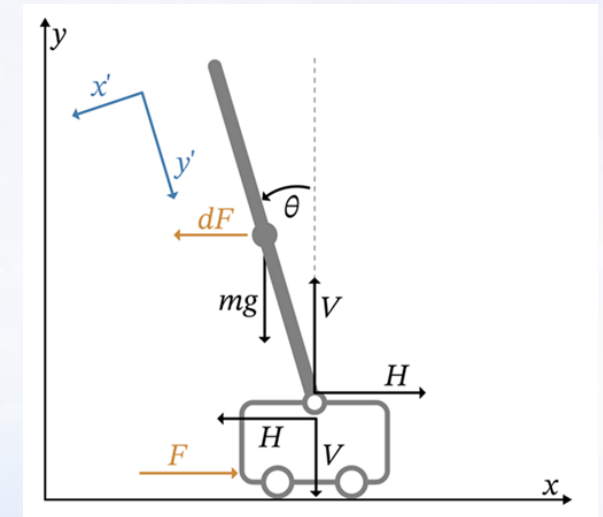
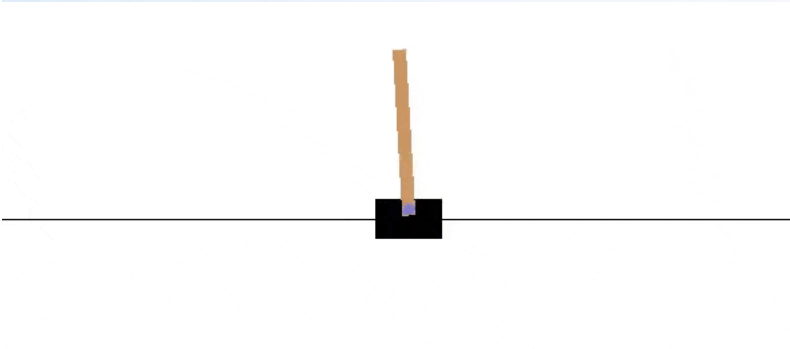
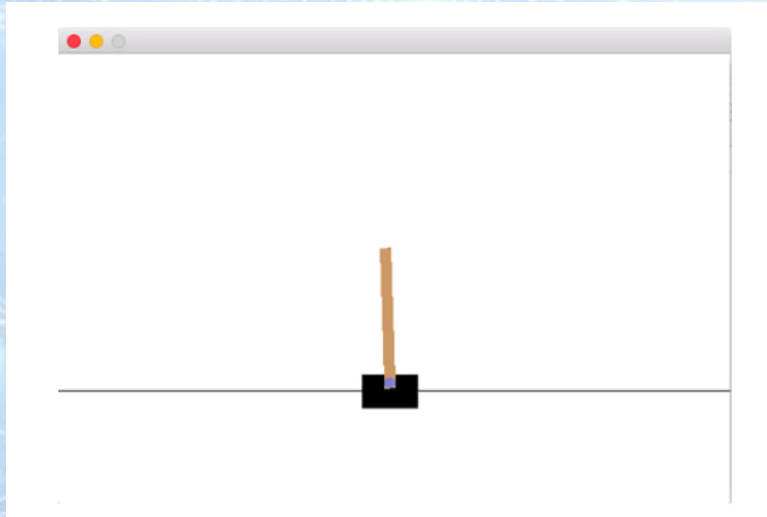
    # 4. Thực hiện hành động trong môi trường
    observation, reward, terminated, truncated, info = env.step(action)

    # 5. Nếu kết thúc, reset lại môi trường
    if terminated or truncated:
        observation, info = env.reset()

# 6. Đóng môi trường
env.close()
```

PhD Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Làm quen môi trường CartPole



Môi trường **CartPole** (trong OpenAI Gym / Gymnasium) là một bài toán điều khiển kinh điển:

- Một **xe đẩy (cart)** di chuyển trái/phải trên một đường thẳng.
- Một **cây gậy (pole)** gắn trên xe.

Mục tiêu: giữ cho cây gậy **không bị ngã** càng lâu càng tốt.

Phiên bản thường dùng: *CartPole-v1*.

PhD Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn

Tài liệu tham khảo

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] C. Szepesvári, *Algorithms for Reinforcement Learning*. San Rafael, CA, USA: Morgan & Claypool Publishers, 2010.
- [3] M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, U.K.: Packt Publishing, 2020.
- [4] L. Serrano, *Grokking Machine Learning*. Shelter Island, NY, USA: Manning Publications, 2021.
- [5] OpenAI, “Spinning Up in Deep Reinforcement Learning,” 2018. [Online]. Available: <https://spinningup.openai.com>
- [6] Farama Foundation, “Gymnasium Documentation,” 2023. [Online]. Available: <https://gymnasium.farama.org>
- [7] DLR-RM, “Stable-Baselines3 Documentation,” 2023. [Online]. Available: <https://stable-baselines3.readthedocs.io>
- [8] DeepMind, “Reinforcement Learning Course Materials,” 2021. [Online]. Available: <https://deepmind.com/learning-resources>
- [9] University of Alberta, “Reinforcement Learning Specialization,” Coursera, 2020. [Online]. Available: <https://www.coursera.org/specializations/reinforcement-learning>